Análisis de Texto con spaCy

Práctica: Búsqueda de Patrones en Percy Jackson

Este notebook demuestra técnicas de Procesamiento de Lenguaje Natural (NLP) usando spaCy para:

- **Descargar y procesar texto de libros**
- **Frecuencia de palabras y estadísticas del texto**
- **Tokenizar y analizar documentos**
- **Buscar patrones específicos usando Matcher**
- **Extraer contexto alrededor de coincidencias**
- **Reconocimiento de Entidades Nombradas (NER)**
- **Análisis de Part-of-Speech (POS)**
- **Extracción de frases nominales y verbales**
- **Análisis de similitud semántica**
- **PhraseMatcher para frases compuestas**

Universidad Icesi

Maestria en IA Aplicada

Catedra: Procesamiento de Lenguaje Natural

Grupo: Golf Realizado por:

- **Arlex Pino**
- **Alfredo Aponte**

---

1. Configuración Inicial

Verificamos si estamos en Google Colab para descargar dependencias si es necesario.

In [1]:

```
import sys
import warnings

warnings.filterwarnings('ignore')

IN_COLAB = 'google.colab' in sys.modules
```

In [2]:

```
if IN_COLAB:
    !wget https://github.com/Ohtar10/icesi-nlp/raw/refs/heads/main/requirements.txt
    %pip install -r requirements.txt
```

---

2. Cargar spaCy

Cargamos el modelo de inglés `en_core_web_sm` que incluye:

- **Tokenizador**
- **Part-of-Speech (POS) tagger**
- **Dependency parser**
- **Named Entity Recognizer (NER)**

In [3]:

```
# RUN THIS CELL to perform standard imports:
import spacy
nlp = spacy.load('en_core_web_sm')
```

---

3. Descargar Dataset

Descargamos los libros de Percy Jackson desde Kaggle y copiamos los archivos a la carpeta actual para facilitar su acceso.

In [4]:

```
import kagglehub

try:
    print("✓ kagglehub ya está instalado")
except ImportError:
    print("Instalando kagglehub...")
    !pip install kagglehub
    print("✓ kagglehub ha sido instalado correctamente")
```

```
✓ kagglehub ya está instalado
```

In [5]:

```
import os
import shutil

# Download latest version
path = kagglehub.dataset_download("shobhit043/percy-jackson-first-5-books")

print("Path to dataset files:", path)
print("Files downloaded:", os.listdir(path))

# Copy files to current directory
for file in os.listdir(path):
    src = os.path.join(path, file)
    dst = os.path.join(".", file)
    if os.path.isfile(src):
        shutil.copy(src, dst)
    elif os.path.isdir(src):
        shutil.copytree(src, dst, dirs_exist_ok=True)

print("Files in current directory:", os.listdir('.'))
```

```
Path to dataset files: C:\Users\apont\.cache\kagglehub\datasets\shobhit043\percy-jackson-fir
st-5-books\versions\1
Files downloaded: ['percy_jackson_book_1.txt', 'percy_jackson_book_2.txt', 'percy_jackson_bo
ok_3.txt', 'percy_jackson_book_4.txt', 'percy_jackson_book_5.txt']
Files in current directory: ['01. El caballero de la armadura oxidada - Robert Fisher.pdf',
'02-26-GQ.pdf', '03-26-Arquitectura y Diseno (1).pdf', '03-26-Arquitectura y Diseno.pdf', '1
5_Diccionario_de_Datos-_SECOP_II_-_Contratos_Electr_nicos.docx', '2025-03-19 Meeting Transcr
iption.pdf', '2025-05-16 Meeting Transcription.pdf', '2025-05-23 Meeting Transcription.pdf',
'2025-06-16 Proyecto II de innovacinnn tecnolnngica en IA.pdf', '2025-06-27 Meeting Transcri
ption.pdf', '20250503 Arboleda Colpatria.pdf', '20250503 Arboleda.PDF', '20250503 Ford BBVA.
pdf', '20250503 Ford Sufi.pdf', '20250503 Ford Wompi.pdf', '20250509 Predial Blue Park SDHR
```

pdf', '20250503 Ford Suff.pdf', '20250503 Ford WOMPI.pdf', '20250509 Predial Blue Park 3DHB.
pdf', '20250509 Predial Zento 75 1102.pdf', '20250703 Predial Blue 23 - Apto.pdf', '20250703
Predial Blue Apto.pdf', '20250707 Pago iPad Alfredo Aponte ePayco.pdf', '20250707 Pago iPad
Alfredo Aponte.pdf', '20250805_200744_939.jpg', '20250805_200842_603.jpg', '20250805_2038031
80.mp4', '20250805_205933672.mp4', '230538.230561.pdf', '3 Conceptos Claves en NLP con Deep
Learning.pdf', '4 Attention is All you Need - GrÃ¡ficamente.pdf', '5 Attention is All you Ne
ed - TÃ©cnicamente.pdf', '682b8bd44d29a7cc439a4a8f.pdf', '682b8d654d29a7cc439a605b.pdf', 'ab
ap_02.csv', 'AGO-2025.pdf', 'Alfredo Aponte Segura_DR 2023_2118617891910.pdf', 'Alfredo_Apon
te_Taller_1.ipynb', 'Anaconda3-2024.10-1-Windows-x86_64.exe', 'Antigravity.exe', 'api-key-HF
33WUSUC52AQUT2.txt', 'Arquitectura v2.drawio.png', 'Arquitectura v3.drawio.png', 'Audio.opus
', 'AZ1 - Gobernanza.pdf', 'basic-miktex-25.12-x64.exe', 'Blue Green and White Clean Renewab
le Energy Presentation.pdf', 'BoardingPass (1).pdf', 'boardingPass (2).pdf', 'BoardingPass.p
df', 'Casos de Uso (1).docx', 'Casos de Uso.docx', 'CCExAssistant-main.zip', 'cce_manual_dat
os_abiertos.pdf', 'Cell 12.csv', 'ChatGPT Installer.exe', 'chrome-win', 'ChromeSetup.exe', '
CitrixWorkspaceApp.exe', 'Claude-Setup-x64.exe', 'client_secret_255572222440-o1gk13bhmok4ae6
ktp66q5llf2i50tni.apps.googleusercontent.com.json', 'CNN.ipynb', 'comet_installer_latest.exe
', 'ComprobantePagoGwClaro_300849646.pdf', 'comprobante_de_pago icesi.pdf', 'comprobante_de_
pago.pdf', 'Comprobante_Pago_TDC.pdf', 'Constancia Matricula Semestre 3.pdf', 'Copia de 2025
0502 Calificaciones Ética y responsabilidad social G1 - MCD.xlsx', 'Coproscópico Mono.PDF',
'CrucialESScan (1).exe', 'CrucialESScan (2).exe', 'CrucialESScan.exe', 'CursorSetup-x64-2.1.
44.exe', 'CursorUserSetup-x64-1.0.0.exe', 'CursorUserSetup-x64-1.5.7.exe', 'DailyForecast_MX
.gz', 'Deja de ser tu - Joe Dispenza.pdf', 'desktop.ini', 'Despliegue_Modelo_XGBoost_Vino.pd
f', 'DexPassG_Architecture_Document.pdf', 'DexPassG_System_Specification.pdf', 'DP900.pptx',
'drive-download-20250530T020114Z-1-001.zip', 'ecosystemFi_Matrix_funding_ope_v1.pdf', 'ejemp
lo_preguntas (1).csv', 'ejemplo_preguntas.csv', 'El modelo circumplejo del afecto_ un enfoqu
e integrador de la neurociencia afectiva, el desarrollo cognitivo y la psicopatología - PMC.
pdf', 'energy_matrix.ipynb', 'epm_free_installer.17470156742474b589.exe', 'Esquire Colombia
- N04 2026_260219_203824 (1).pdf', 'Esquire Colombia - N04 2026_260219_203824 (2).pdf', 'Esq
uire Colombia - N04 2026_260219_203824.pdf', 'Exam Ref AZ-104 Microsoft Azure Administrator
- Charles Pluta.pdf', 'ExtractoOccidente.pdf', 'FEB-2025.pdf', 'Grabaciónes_Procesamiento_nu
be_2 -20250110T042952Z-003', 'Grupos_SR_2025_2.xlsx', 'Guia_Despliegue_Modelo_XGBoost_Vino_D
etallado.pdf', 'Guia_Proyecto_Prediccion_Series_AAPL.pdf', 'horarios medicinas CHIQUITINES.x
lsx', 'Icesi_para_diapositivas.pptx.pdf', 'id_service_TEF.pdf', 'Imagen de WhatsApp 2025-06-
20 a las 11.11.32_0974466b.jpg', 'Informe_Examen_Mono.pdf', 'Infrati_Notas_2024-1.xlsx', 'in
stall-tl-windows.exe', 'Intro Seminario.pptx', 'kubectl', 'kubernetes.drawio', 'Lenovo Repai
r Checklist.pdf', 'M716_Shock_Absorbers_Catalogue_2019_10.pdf', 'Machine Learning, Pipelines
, Deployment and MLOps Tutorial _ DataCamp.pdf', 'Maestria_IA_Semestre_III.ics', 'MagicCompR
ules 20250725.txt', 'MAR-2025.pdf', 'Miniconda3-latest-Windows-x86_64.exe', 'MLOps-20250608T
011505Z-1-001.zip', 'Music Recommender Systems.pdf', 'nihms44490.pdf', 'node-v22.19.0-x64.ms
i', 'Pago Factura 00249993.jpg', 'Parqueadero Moto.pdf', 'Parqueadero.pdf', 'Parte 2.mp4', '
PDG-Musical 2025 v1.docx', 'PDG-Musical 2025 v10.docx', 'PDG-Musical 2025 v11.docx', 'PDG-Mu
sical 2025 v12.docx', 'PDG-Musical 2025 v2.docx', 'PDG-Musical 2025 v3.docx', 'PDG-Musical 2
025 v4.docx', 'PDG-Musical 2025 v5.docx', 'PDG-Musical 2025 v6.docx', 'PDG-Musical 2025.docx
', 'percy_jackson_book_1.txt', 'percy_jackson_book_2.txt', 'percy_jackson_book_3.txt', 'perc
y_jackson_book_4.txt', 'percy_jackson_book_5.txt', 'PlanDePagos.pdf', 'poppler-windows-24.08
.0-0.zip', 'Portadas México 210226.pdf', 'Preguntas_Medico_Luxacion_Hombro.pdf', 'processing
_xgboost.ipynb', 'PS1 - Commandlets y Modulos.pdf', 'PS2 - Procesos y Servicios.pdf', 'PS3 -
WMI y Pipes.pdf', 'PUJ', 'Python-3.11.13.tar.xz', 'Python-3.11.13.tgz', 'P_BTPA_2408 - OCT25
- copia.xlsx', 'P_BTPA_2408 - OCT25.xlsm', 'P_BTPA_2408 - OCT25.xlsx', 'P_BTPA_2408 - OCT25v
2.xlsx', 'Q2l0cml4X0Nsb3VkLkFNRVIgU0FQIERlc2t0b3AgJFM4MzItNDc5.ica', 'quick-start-minimal.ya
ml', 'quizz_perp.html', 'quiz_app v2.html', 'quiz_app.html', 'quiz_template.xlsx', 'README.m
d', 'README_Tesis_FWOD.pdf', 'Release-24.08.0-0.zip', 'requirements.txt', 'Revista Seleções-
Fev 26.pdf', 'Revista Vogue - Fev 26.pdf', 'sample.csv', 'SECOP_II_-_Contratos_Electrónicos_
20260130.csv', 'Sol-Doc-v1.pdf', 'Sol_Arch_ecosystemFi_v1.pdf', 'Tesis.pptx', 'tesis_diapos.
pdf', 'TokenizationService_TEF.pdf', 'Torre 4 Apto 1013 Parqueadero Moto.pdf', 'Torre 4 Apto
1013 Parqueadero.pdf', 'Tu pasabordo (1).pdf', 'Tu pasabordo.pdf', 'U-Icesi', 'Un Sistema Re
comendador Inteligente para Universidades.pdf', 'UnixLocalAccounts Module.ps1', 'url_clase_1
.txt', 'url_clase_2 (1).txt', 'url_clase_2.txt', 'WhatsApp Image 2026-01-19 at 3.46.44 PM.jp
eg', 'windows_10_cmake_Release_Graphviz-13.0.1-win64.zip', 'Work 6 practice.ipynb', 'Work 6
practice_spacy_enriquecido.ipynb', 'Work 7 sentiment-analysis_Grupo_Golf.ipynb', 'Work_6_pra
ctice_Grupo_Golf.ipynb', 'Work_6_practice_Grupo_Golf.pdf', 'Work_7_sentiment-analysis_enrich
ed.ipynb', 'y_test Vs y_pred.csv', 'Zerotier ID.txt', 'ZeroTier One.msi']

---

4. Procesar el Documento

Leemos el primer libro de Percy Jackson y lo procesamos con spaCy. El objeto `doc` contiene:

- **Tokens individuales**
- **Información gramatical (POS tags, dependencias)**
- **Entidades nombradas**
- **Estructura de oraciones**

In [6]:

```
# Leer el archivo y procesarlo con spaCy
with open('./percy_jackson_book_1.txt', encoding='utf-8') as file:
    doc = nlp(file.read())

print(f"Documento procesado: {len(doc)} tokens")
```

```
Documento procesado: 127898 tokens
```

4.1. Vista Previa del Documento

Mostramos los primeros 50 tokens para verificar que el documento se cargó correctamente.

In [7]:

```
# Primeros 50 tokens del documento
doc[:50]
```

Out[7]:

```


BOOKS BY RICK RIORDAN

PERCY JACKSON AND THE OLYMPIANS
The Lightning Thief
The Sea of Monsters
The Titan’s Curse
The Battle of the Labyrinth
The Last Olympian
The Demigod Files
Percy Jackson’s Greek Gods, illustrated by John Rocco
```

4.2. Estadísticas Generales y Riqueza del Vocabulario

Analizamos características cuantitativas del texto para entender el estilo del autor:

- **Type-Token Ratio (TTR): mide la riqueza del vocabulario (más alto = más variado)**
- **Densidad léxica: proporción de palabras con contenido semántico**
- **Longitud promedio de oración: indica complejidad narrativa**

In [8]:

```
# ==============================================
# ESTADÍSTICAS GENERALES DEL TEXTO
# is_alpha  → solo letras (sin puntuación ni números)
# is_stop   → palabras vacías (the, a, is...)
# is_punct  → puntuación
# ==============================================

# Tokens básicos
total_tokens = len(doc)
tokens_alpha = [t for t in doc if t.is_alpha]
tokens_sin_stopwords = [t for t in tokens_alpha if not t.is_stop]
palabras_unicas = set(t.lemma_.lower() for t in tokens_alpha)
```

```
# Oraciones
total_oraciones = len(list(doc.sents))
longitudes_oraciones = [len([t for t in s if t.is_alpha]) for s in doc.sents]
promedio_oracion = sum(longitudes_oraciones) / len(longitudes_oraciones)

# Métricas de vocabulario
ttr = len(palabras_unicas) / len(tokens_alpha) * 100  # Type-Token Ratio
densidad_lexica = len(tokens_sin_stopwords) / len(tokens_alpha) * 100

print("=" * 60)
print("   ESTADÍSTICAS DEL LIBRO 1 - PERCY JACKSON")
print("=" * 60)
print(f"\n  Tokens totales:             {total_tokens:>10,}")
print(f"  - Palabras (solo letras):   {len(tokens_alpha):>10,}")
print(f"  - Vocabulario único:        {len(palabras_unicas):>10,}")
print(f"  - Total de oraciones:       {total_oraciones:>10,}")
print(f"\n  MÉTRICAS DE ESTILO:")
print(f"  - Type-Token Ratio (TTR):      {ttr:>9.2f}%")
print(f"  - Densidad léxica:             {densidad_lexica:>9.2f}%")
print(f"  - Longitud promedio oración:    {promedio_oracion:>9.1f} palabras")
print("=" * 60)
```

```
============================================================
   ESTADÍSTICAS DEL LIBRO 1 - PERCY JACKSON
============================================================

  Tokens totales:                127,898
  - Palabras (solo letras):       92,026
  - Vocabulario único:             5,764
  - Total de oraciones:            8,946

  MÉTRICAS DE ESTILO:
  - Type-Token Ratio (TTR):           6.26%
  - Densidad léxica:                 44.38%
  - Longitud promedio oración:         10.3 palabras
============================================================
```

---

5. Búsqueda de Patrones con Matcher

Usamos `Matcher` de spaCy para encontrar la frase "peanut butter" en el documento.

¿Cómo funciona?

- **Definimos un patrón: dos tokens consecutivos con texto "peanut" y "butter" (en minúsculas)**
- **El Matcher busca todas las ocurrencias en el documento**
- **Devuelve tuplas `(match_id, start, end)` con la posición de cada coincidencia**

In [9]:

```
from spacy.matcher import Matcher

matcher = Matcher(nlp.vocab)
pattern = [{'LOWER': 'peanut'}, {'LOWER': 'butter'}]
matcher.add("like", [pattern])

found_matches = matcher(doc)
found_matches
```

Out[9]:

```
[(18194338103975822726, 1326, 1328), (18194338103975822726, 1551, 1553)]
```

5.1 Contexto de las Coincidencias

Veamos el contexto alrededor de cada coincidencia (tokens antes y después) para entender mejor el uso.

In [10]:

```
start, end = found_matches[0][1:]
doc[start-9:end+13]
```

Out[10]:

```
the back of the


head with chunks of peanut butter-and-ketchup sandwich.
Grover was an easy target.
```

In [11]:

```
start, end = found_matches[1][1:]
doc[start-7:end+5]
```

Out[11]:

```
“It’s okay. I like peanut butter.”
He dodged
```

5.2 Oraciones Completas

Extraemos las oraciones completas que contienen las coincidencias. Esto proporciona el contexto completo y gramaticalmente correcto.

In [12]:

```
# Primero obtenemos todas las oraciones del documento
sentences = list(doc.sents)

print(f"Oraciones que contienen 'peanut butter':\n")

# Buscamos cada coincidencia en las oraciones
for i, (_, start, end) in enumerate(found_matches, 1):
    for sentence in sentences:
        # Verificar si la coincidencia está dentro de esta oración
        if sentence.start <= start and sentence.end >= end:
            print(f"{i}. {sentence.text.strip()}\n")
```

```
Oraciones que contienen 'peanut butter':

1. All the way into the city, I put up with Nancy Bobofit, the freckly,
redheaded kleptomaniac girl, hitting my best friend Grover in the back of the

head with chunks of peanut butter-and-ketchup sandwich.

2. I like peanut butter.”
```

---

6. Reconocimiento de Entidades Nombradas (NER)

spaCy puede identificar automáticamente entidades nombradas en el texto como:

- **PERSON: nombres de personas**
- **GPE: países, ciudades, estados**

- **ORG: organizaciones**
- **LOC: lugares geográficos**
- **DATE: fechas**

Esto es muy útil para entender qué personajes, lugares y organizaciones son más importantes en el libro.

In [13]:

```
from collections import Counter
# ==============================================
# RECONOCIMIENTO DE ENTIDADES NOMBRADAS (NER)
# doc.ents contiene todas las entidades detectadas
# Cada entidad tiene: .text (texto), .label_ (tipo), .start/.end (posición)
# ==============================================

# Extraemos todas las entidades del documento
entidades = [(ent.text.strip(), ent.label_) for ent in doc.ents]

print(f"Total de entidades encontradas: {len(entidades)}\n")

# Agrupamos por tipo de entidad
tipos = Counter([label for _, label in entidades])
print("Distribución por tipo de entidad:")
for tipo, cantidad in tipos.most_common():
    print(f"  {tipo}: {cantidad} ocurrencias")
```

```
Total de entidades encontradas: 4041

Distribución por tipo de entidad:
  PERSON: 1515 ocurrencias
  ORG: 692 ocurrencias
  CARDINAL: 488 ocurrencias
  DATE: 303 ocurrencias
  GPE: 263 ocurrencias
  NORP: 150 ocurrencias
  ORDINAL: 116 ocurrencias
  WORK_OF_ART: 107 ocurrencias
  TIME: 96 ocurrencias
  FAC: 81 ocurrencias
  LOC: 78 ocurrencias
  QUANTITY: 65 ocurrencias
  PRODUCT: 43 ocurrencias
  LANGUAGE: 15 ocurrencias
  MONEY: 12 ocurrencias
  EVENT: 10 ocurrencias
  LAW: 4 ocurrencias
  PERCENT: 3 ocurrencias
```

6.1. Top de 15 personajes mas mencionados

Filtramos solo las entidades de tipo PERSON y contamos cuáles aparecen con más frecuencia

In [14]:

```
personas = [ent.text.strip() for ent in doc.ents if ent.label_ == 'PERSON']
conteo_personas = Counter(personas)

print("Top 15 personajes más mencionados en el Libro 1:")
print("-" * 45)
for nombre, veces in conteo_personas.most_common(15):
    print(f"  {nombre:<25} → {veces} menciones")
```

```
Top 15 personajes más mencionados en el Libro 1:
---------------------------------------------
  Annabeth                  → 220 menciones
  Percy                     → 178 menciones
```

```
  Grover                    → 148 menciones
  Luke                      → 111 menciones
  Zeus                      → 104 menciones
  Poseidon                  → 65 menciones
  Dodds                     → 59 menciones
  Brunner                   → 54 menciones
  D                         → 43 menciones
  Percy Jackson             → 38 menciones
  Clarisse                  → 33 menciones
  Kronos                    → 27 menciones
  Charon                    → 20 menciones
  Aunty Em                  → 19 menciones
  Cerberus                  → 19 menciones
```

6.1.1. Interpretación de Personajes Detectados (PERSON)

La entidad PERSON identifica nombres de personas, tanto reales como ficticias. En el contexto de Percy Jackson:

Protagonistas y semidioses:

- **Los nombres que aparecen con más frecuencia suelen ser los personajes principales de la narrativa**
- **Percy Jackson es el protagonista (hijo de Poseidón)**
- **Otros semidioses importantes como Annabeth (hija de Atenea), Grover (sátiro)**

Dioses olímpicos:

- **Zeus, Poseidón, Hades (los tres grandes)**
- **Atenea, Ares, Hermes, etc.**
- **Pueden aparecer como PERSON o como entidades mitológicas**

Personajes secundarios:

- **Profesores, mentores (Quirón)**
- **Antagonistas y villanos**
- **Mortales que interactúan con el protagonista**

> **Análisis:** **La frecuencia de mención indica la importancia del personaje en la trama. Si un nombre aparece cientos de veces, es un personaje central.**

6.2. Top de Lugares mencionados

In [15]:

```python
# ============================================
# LUGARES Y ORGANIZACIONES MÁS MENCIONADOS
# GPE = países, ciudades, estados
# LOC = lugares geográficos generales
# ORG = organizaciones
# ============================================

lugares = [ent.text.strip() for ent in doc.ents if ent.label_ in ('GPE', 'LOC')]

print("Top 10 lugares mencionados:")
print("-" * 35)
for lugar, veces in Counter(lugares).most_common(10):
    print(f"  {lugar:<25} → {veces} veces")
```

```
Top 10 lugares mencionados:
-----------------------------------
  New York                  → 17 veces
  Hermes                    → 16 veces
  Thalia                    → 15 veces
```

```
  Denver                    → 12 veces
  Grover                    → 11 veces
  Los Angeles               → 11 veces
  L.A.                      → 9 veces
  Manhattan                 → 8 veces
  Poseidon                  → 8 veces
  Magnus                    → 8 veces
```

6.2. Top de Organizaciones mencionadas

In [16]:

```
orgs = [ent.text.strip() for ent in doc.ents if ent.label_ == 'ORG']
print("\nTop 10 organizaciones mencionadas:")
print("-" * 35)
for org, veces in Counter(orgs).most_common(10):
    print(f"  {org:<25} → {veces} veces")
```

```
Top 10 organizaciones mencionadas:
-----------------------------------
  Chiron                    → 183 veces
  Olympus                   → 35 veces
  Athena                    → 31 veces
  Medusa                    → 18 veces
  Underworld                → 15 veces
  Yancy Academy             → 14 veces
  Hades                     → 12 veces
  Pan                       → 12 veces
  Hephaestus                → 11 veces
  Furies                    → 11 veces
```

---

7. Análisis de Part-of-Speech (POS)

El etiquetado POS (Part of Speech) clasifica cada token según su función gramatical:

- **NOUN: sustantivos**
- **VERB: verbos**
- **ADJ: adjetivos**
- **ADV: adverbios**
- **PROPN: nombres propios**

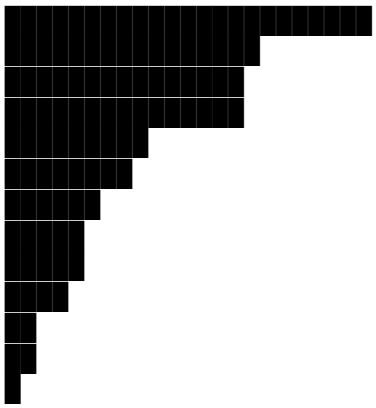
Esto nos ayuda a entender el vocabulario y el estilo narrativo del autor.

In [17]:

```
# ==============================================
# ANÁLISIS PART-OF-SPEECH (POS)
# token.pos_  → categoría gramatical general (NOUN, VERB, ADJ...)
# token.tag_  → etiqueta detallada (NNS, VBD, JJ...)
# token.lemma_ → forma base o raíz de la palabra
# ==============================================

# Contamos la frecuencia de cada categoría gramatical
pos_counts = Counter([token.pos_ for token in doc if not token.is_space])

print("Distribución de categorías gramaticales en el Libro 1:")
print("-" * 45)
for pos, count in pos_counts.most_common():
    barra = '▇' * (count // 1000)
    print(f"  {pos:<10} {count:>7} tokens  {barra}")
```

```
Distribución de categorías gramaticales en el Libro 1:
```

```
Distribución de categorías gramaticales en el libro 1:
----------------------------------------------
  PUNCT        23748 tokens
  NOUN         16132 tokens
  VERB         15446 tokens
  PRON         15328 tokens
  ADP           9103 tokens
  DET           8143 tokens
  AUX           6113 tokens
  ADJ           5518 tokens
  PROPN         5475 tokens
  ADV           4957 tokens
  PART          2821 tokens
  CCONJ         2798 tokens
  SCONJ         1889 tokens
  NUM            884 tokens
  INTJ           599 tokens
  SYM             25 tokens
  X               13 tokens
```



7.1. Verbos mas usados

In [18]:

```
# ============================================
# TOP VERBOS Y ADJETIVOS MÁS USADOS
# Usamos token.lemma_ para agrupar formas del mismo verbo
# (ej: "ran", "run", "running" → "run")
# Filtramos stop words y puntuación con is_stop e is_alpha
# ============================================

# Verbos más frecuentes (en su forma base/lema)
verbos = [
    token.lemma_.lower()
    for token in doc
    if token.pos_ == 'VERB' and token.is_alpha and not token.is_stop
]

print("Top 15 verbos más usados (forma base):")
print("-" * 35)
for verbo, veces in Counter(verbos).most_common(15):
    print(f"  {verbo:<20} → {veces} veces")
```

```
Top 15 verbos más usados (forma base):
-----------------------------------
  say                  → 721 veces
  look                 → 401 veces
  know                 → 292 veces
  tell                 → 282 veces
  get                  → 271 veces
  think                → 245 veces
  go                   → 233 veces
  come                 → 232 veces
  want                 → 205 veces
  try                  → 168 veces
  feel                 → 157 veces
  see                  → 141 veces
  ask                  → 140 veces
  turn                 → 139 veces
  start                → 119 veces
```

7.2. Adjetivos mas usados

In [19]:

```
# Adjetivos más frecuentes
```

```
# Adjetivos más frecuentes
adjetivos = [
    token.lemma_.lower()
    for token in doc
    if token.pos_ == 'ADJ' and token.is_alpha and not token.is_stop
]



print("\nTop 15 adjetivos más usados:")
print("-" * 35)
for adj, veces in Counter(adjetivos).most_common(15):
    print(f"  {adj:<20} → {veces} veces")
```

```
Top 15 adjetivos más usados:
-----------------------------------
  good                 → 149 veces
  little               → 115 veces
  old                  → 96 veces
  big                  → 86 veces
  right                → 85 veces
  bad                  → 75 veces
  sure                 → 71 veces
  black                → 69 veces
  dead                 → 60 veces
  long                 → 49 veces
  huge                 → 48 veces
  well                 → 43 veces
  red                  → 41 veces
  new                  → 38 veces
  blue                 → 38 veces
```

---

8. Extracción de Frases Nominales y Verbales (Chunks)

spaCy permite extraer automáticamente frases nominales (`noun_chunks`) y frases verbales del texto.

- **Noun chunks: grupos de palabras que funcionan como sustantivos ("the big blue sea")**
- **Son útiles para identificar conceptos clave y objetos importantes en la narrativa**

Cada chunk tiene:

- `.text`**: el texto completo de la frase**
- `.root`**: la palabra principal del grupo**
- `.root.dep_`**: la dependencia sintáctica del núcleo**

In [20]:

```
# ==============================================
# FRASES NOMINALES (NOUN CHUNKS)
# doc.noun_chunks itera sobre todas las frases nominales del documento
# Cada chunk es un Span con atributos: text, root, root.dep_, root.head
# ==============================================

# Extraemos todas las frases nominales
noun_chunks = list(doc.noun_chunks)
print(f"Total de frases nominales encontradas: {len(noun_chunks)}\n")

# Contamos las más frecuentes (normalizamos a minúsculas)
chunk_texts = [chunk.text.lower().strip() for chunk in noun_chunks]
chunk_counter = Counter(chunk_texts)

print("Top 20 frases nominales más frecuentes:")
```

```
print("-" * 45)
for chunk, veces in chunk_counter.most_common(20):
    print(f"  '{chunk}'  → {veces} veces")
```

```
Total de frases nominales encontradas: 29117

Top 20 frases nominales más frecuentes:
---------------------------------------------
  'i'  → 3521 veces
  'you'  → 1314 veces
  'me'  → 1035 veces
  'he'  → 1014 veces
  'it'  → 995 veces
  'she'  → 637 veces
  'we'  → 584 veces
  'that'  → 476 veces
  'grover'  → 401 veces
  'annabeth'  → 378 veces
  'they'  → 375 veces
  'what'  → 301 veces
  'him'  → 293 veces
  'us'  → 207 veces
  'her'  → 193 veces
  'who'  → 177 veces
  'chiron'  → 168 veces
  'something'  → 164 veces
  'them'  → 157 veces
  'this'  → 96 veces
```

---

9. Análisis de Similitud Semántica

spaCy puede calcular la similitud semántica entre palabras o textos usando vectores de palabras (word vectors).

La similitud va de 0 a 1 donde:

- **1.0 = idéntico**
- **0.8+ = muy similar**
- **0.5-0.7 = relacionado**
- **< 0.3 = poco relacionado**

In [21]:

```
# ============================================
# SIMILITUD SEMÁNTICA ENTRE PALABRAS
# token.similarity(otro_token) compara vectores de palabras
# Útil para encontrar palabras relacionadas temáticamente
# ============================================

# Comparamos palabras clave del universo de Percy Jackson
palabras_clave = ['Percy', 'hero', 'monster', 'god', 'water', 'sword', 'camp', 'quest']
tokens_clave = [nlp(palabra)[0] for palabra in palabras_clave]

print("Matriz de Similitud Semántica entre palabras clave:")
print("(Valor entre 0 y 1, donde 1 = idéntico)\n")

# Encabezado de la tabla
header = f"{'':>10}" + "".join(f"{w:>10}" for w in palabras_clave)
print(header)
print("-" * len(header))

# Filas de similitud
for t1 in tokens_clave:
```

```
    fila = f"{t1.text:>10}"
    for t2 in tokens_clave:
        sim = t1.similarity(t2)
        fila += f"  {sim:.2f}   "
    print(fila)
```

```
Matriz de Similitud Semántica entre palabras clave:
(Valor entre 0 y 1, donde 1 = idéntico)

               Percy      hero   monster       god     water     sword      camp     quest
-----------------------------------------------------------------------------------------
     Percy  1.00     0.52     0.39     0.47     0.61     0.54     0.60     0.42
      hero  0.52     1.00     0.44     0.36     0.61     0.69     0.50     0.45
   monster  0.39     0.44     1.00     0.41     0.59     0.47     0.48     0.59
       god  0.47     0.36     0.41     1.00     0.64     0.52     0.51     0.33
     water  0.61     0.61     0.59     0.64     1.00     0.78     0.73     0.49
     sword  0.54     0.69     0.47     0.52     0.78     1.00     0.64     0.45
      camp  0.60     0.50     0.48     0.51     0.73     0.64     1.00     0.52
     quest  0.42     0.45     0.59     0.33     0.49     0.45     0.52     1.00
```

---

10. PhraseMatcher para Búsqueda de Frases Compuestas

`PhraseMatcher` **es más eficiente que** `Matcher` **cuando queremos buscar una lista de frases específicas.**

Es ideal para:

- **Buscar múltiples nombres de personajes a la vez**
- **Identificar términos mitológicos**
- **Detectar lugares del universo de Percy Jackson**

In [22]:

```
from spacy.matcher import PhraseMatcher

# ==============================================
# PHRASEMATCHER - BÚSQUEDA EFICIENTE DE FRASES
# A diferencia del Matcher, el PhraseMatcher trabaja
# directamente con objetos Doc procesados por spaCy
# attr='LOWER' hace la búsqueda insensible a mayúsculas
# ==============================================

phrase_matcher = PhraseMatcher(nlp.vocab, attr='LOWER')

# Definimos grupos temáticos de frases a buscar
personajes_olimpicos = [
    "Zeus", "Poseidon", "Hades", "Athena", "Ares", "Hermes",
    "Apollo", "Artemis", "Hephaestus", "Aphrodite", "Dionysus"
]

lugares_magicos = [
    "Mount Olympus", "the Underworld", "Camp Half-Blood",
    "the Empire State Building", "the Sea of Monsters"
]

# Convertimos las frases a objetos Doc y las añadimos al matcher
patrones_dioses = [nlp.make_doc(texto) for texto in personajes_olimpicos]
patrones_lugares = [nlp.make_doc(texto) for texto in lugares_magicos]

phrase_matcher.add("DIOSES_OLIMPICOS", patrones_dioses)
phrase_matcher.add("LUGARES_MAGICOS", patrones_lugares)

# Buscamos en el documento
matches_frases = phrase_matcher(doc)
```

```python
print(f"Total de coincidencias encontradas: {len(matches_frases)}\n")

# Contamos por categoría
conteo_dioses = Counter()
conteo_lugares = Counter()

for match_id, start, end in matches_frases:
    categoria = nlp.vocab.strings[match_id]
    texto = doc[start:end].text
    if categoria == 'DIOSES_OLIMPICOS':
        conteo_dioses[texto] += 1
    else:
        conteo_lugares[texto] += 1

print("Dioses olímpicos mencionados:")
print("-" * 35)
for dios, veces in conteo_dioses.most_common():
    print(f"  {dios:<20} → {veces} veces")

print("\nLugares mágicos mencionados:")
print("-" * 35)
for lugar, veces in conteo_lugares.most_common():
    print(f"  {lugar:<30} → {veces} veces")
```

```
Total de coincidencias encontradas: 619

Dioses olímpicos mencionados:
-----------------------------------
  Ares                 → 123 veces
  Hades                → 114 veces
  Zeus                 → 107 veces
  Poseidon             → 89 veces
  Athena               → 34 veces
  Hermes               → 20 veces
  Dionysus             → 18 veces
  Aphrodite            → 13 veces
  Hephaestus           → 13 veces
  Apollo               → 11 veces
  Artemis              → 5 veces

Lugares mágicos mencionados:
-----------------------------------
  the Underworld                 → 39 veces
  Camp Half-Blood                → 14 veces
  Mount Olympus                  → 8 veces
  The Sea of Monsters            → 5 veces
  the Empire State Building      → 3 veces
  CAMP HALF-BLOOD                → 2 veces
  The Underworld                 → 1 veces
```

In [23]:

```python
# ==============================================
# PALABRAS MÁS IMPORTANTES DEL TEXTO
# Excluimos stop words y puntuación para quedarnos
# solo con las palabras con contenido semántico
# Usamos el lemma para agrupar variantes de una misma palabra
# ==============================================

palabras_contenido = [
    token.lemma_.lower()
    for token in doc
    if token.is_alpha
    and not token.is_stop
    and len(token.text) > 2  # Excluimos palabras muy cortas
    and token.pos_ in ('NOUN', 'VERB', 'ADJ', 'PROPN')
```

```
]

print("Top 25 palabras más importantes del libro (sin stop words):")
print("-" * 50)
print(f"{'PALABRA':<20} {'FRECUENCIA':>10} {'BARRA'}")
print("-" * 50)

for palabra, freq in Counter(palabras_contenido).most_common(25):
    barra = '▇' * (freq // 50)
    print(f"  {palabra:<18} {freq:>8}   {barra}")
```

```
Top 25 palabras más importantes del libro (sin stop words):
--------------------------------------------------
PALABRA              FRECUENCIA BARRA
--------------------------------------------------
  say                     721   ▇▇▇▇▇▇▇▇▇▇▇▇▇▇
  grover                  468   ▇▇▇▇▇▇▇▇▇
  look                    435   ▇▇▇▇▇▇▇▇▇
  annabeth                419   ▇▇▇▇▇▇▇▇▇
  know                    292   ▇▇▇▇▇▇
  tell                    282   ▇▇▇▇▇▇
  get                     271   ▇▇▇▇▇▇
  percy                   269   ▇▇▇▇▇▇
  think                   245   ▇▇▇▇▇
  go                      233   ▇▇▇▇▇
  come                    232   ▇▇▇▇▇
  want                    205   ▇▇▇▇▇
  god                     194   ▇▇▇▇
  time                    193   ▇▇▇▇
  chiron                  187   ▇▇▇▇
  eye                     175   ▇▇▇▇
  try                     171   ▇▇▇▇
  feel                    158   ▇▇▇▇
  good                    153   ▇▇▇▇
  turn                    143   ▇▇▇
  see                     141   ▇▇▇
  ask                     140   ▇▇▇
  thing                   139   ▇▇▇
  hand                    133   ▇▇▇
  head                    131   ▇▇▇
```

In [24]:

```
# ==============================================
# TIEMPOS VERBALES PREDOMINANTES EN EL LIBRO
# Analizamos en qué tiempo está escrita la narrativa
# VBD = past tense, VBZ = present, VBP = present non-3rd person
# ==============================================

tiempos_verbales = Counter()

# Mapeo de etiquetas POS detalladas a descripciones legibles
descripcion_tiempo = {
    'VBD': 'Pasado simple',
    'VBZ': 'Presente (3ra persona)',
    'VBP': 'Presente (no 3ra persona)',
    'VBN': 'Participio pasado',
    'VBG': 'Gerundio (-ing)',
    'VB':  'Infinitivo',
    'MD':  'Modal (would, could, can...)'
}

for token in doc:
    if token.tag_ in descripcion_tiempo:
        tiempos_verbales[token.tag_] += 1

print("Distribución de tiempos verbales en el Libro 1:")
```

```
print("-" * 50)

total_verbos = sum(tiempos_verbales.values())
for tag, count in tiempos_verbales.most_common():
    desc = descripcion_tiempo.get(tag, tag)
    porcentaje = count / total_verbos * 100
    barra = '▇' * int(porcentaje / 2)
    print(f"  {desc:<35} {porcentaje:>5.1f}%  {barra}")

print("\n→ Un alto porcentaje de 'Pasado simple' indica narración en pasado.")
```

```
Distribución de tiempos verbales en el Libro 1:
--------------------------------------------------
  Pasado simple                         43.0%  ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇
  Infinitivo                            18.9%  ▇▇▇▇▇▇▇▇▇▇
  Gerundio (-ing)                       10.0%  ▇▇▇▇▇
  Participio pasado                      8.1%  ▇▇▇▇
  Presente (no 3ra persona)              7.6%  ▇▇▇
  Modal (would, could, can...)           7.5%  ▇▇▇
  Presente (3ra persona)                 4.8%  ▇▇

→ Un alto porcentaje de 'Pasado simple' indica narración en pasado.
```

10.1. Lemmatización

La lematización nos permite reducir las palabras a su forma base (lemma), lo que es muy útil para análisis semánticos y de frecuencia.

In [25]:

```
# Extraer lemas únicos más frecuentes (ignorando puntuación y stop words)

from collections import Counter

lemmas = [
    token.lemma_.lower()
    for token in doc
    if not token.is_punct and not token.is_space and not token.is_stop
]

lemma_counts = Counter(lemmas)
top_lemmas = lemma_counts.most_common(20)

print("Lemas más frecuentes (sin stopwords):\n")
for lemma, freq in top_lemmas:
    print(f"{lemma:20} {freq}")
```

```
Lemas más frecuentes (sin stopwords):

say                  721
grover               468
like                 457
look                 435
annabeth             423
know                 292
get                  283
tell                 282
percy                269
think                245
come                 233
go                   233
want                 205
god                  194
time                 193
right                187
chiron               187
```

```
eye                 175
try                 171
feel                158
```

---

11. Visualización de dependencias sintácticas

spaCy incluye `displacy`, una herramienta para dibujar:

- **Las relaciones de dependencia entre palabras**
- **La estructura gramatical de una oración**

Esto ayuda a entender visualmente cómo se organiza la oración.

In [26]:

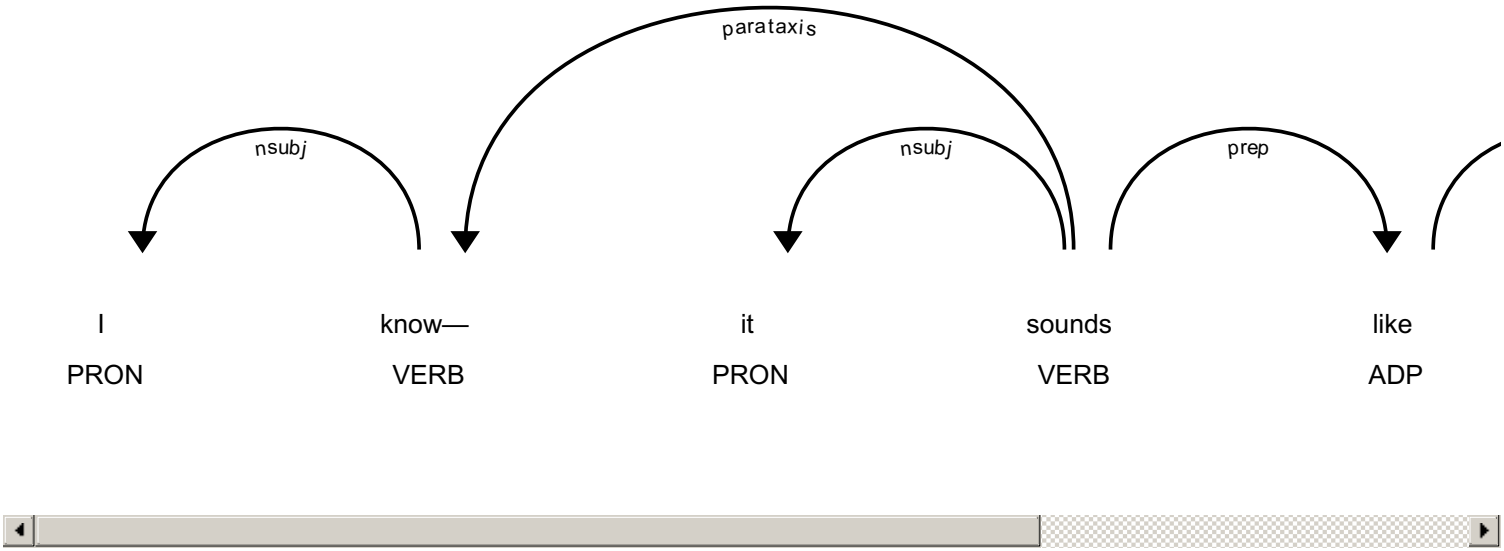
```
from spacy import displacy

# Tomamos una oración representativa del documento
sent_example = list(doc.sents)[50]
sent_example
```

Out[26]:

```
I know—it sounds like torture.
```

In [27]:

```
# Visualizar dependencias sintácticas en Jupyter
displacy.render(sent_example, style="dep", jupyter=True)
```



---

12. Visualización de entidades nombradas (NER)

También podemos resaltar en colores las entidades nombradas detectadas:

- **PERSON (personas)**
- **GPE (países/ciudades)**
- **ORG (organizaciones)**
- **DATE, MONEY, etc.**

Esto da una vista rápida de los elementos importantes del texto.

Esto da una **vista rápida** de los elementos importantes del texto.

In [28]:

```
# Tomamos un fragmento de texto (por ejemplo, las primeras 3 páginas aproximadamente)
text_fragment = doc[:200]  # ajusta el número de tokens si quieres más o menos texto

displacy.render(text_fragment, style="ent", jupyter=True)
```

BOOKS BY RICK RIORDAN PERSON

PERCY JACKSON PERSON AND THE OLYMPIANS

The Lightning Thief The Sea of Monsters WORK_OF_ART

The Titan's Curse

The Battle of the Labyrinth The Last Olympian The Demigod Files Percy Jackson's Greek Gods WORK_OF_ART ,

illustrated by John Rocco PERSON

The Lightning Thief: The Graphic Novel The Sea of Monsters: The Graphic Novel WORK_OF_ART

The Titan's Curse: The Graphic Novel

THE KANE CHRONICLES

The Red Pyramid

The Throne of Fire

The Serpent's Shadow

The Kane Chronicles Survival Guide

The Kane Chronicles Survival Guide (Interactive Version)

The Red Pyramid: The Graphic Novel WORK_OF_ART

The Throne of Fire: The Graphic Novel (coming October 2015 DATE !)

THE HEROES ORG OF OLYMPUS

The Lost Hero The Son of Neptune The Mark of Athena WORK_OF_ART

The House of Hades The Blood of Olympus The Demigod Diaries ORG

The Lost Hero: The Graphic Novel WORK_OF_ART

The Son of Neptune: The Graphic Novel WORK_OF_ART SHORT STORIES BY RICK RIORDAN PERSON

The Son of Sobek WORK_OF_ART

A Carter Kane/Percy